



# An Embedded System Component Network Architecture

*Dan Bouvier*

*System Architecture Manager*

*Motorola, Semiconductor Product Sector*

*Chair, RapidIO Technical Working Group*

## **Abstract**

This paper describes *RapidIO*, a high performance low pin count packet switched system level interconnect architecture. The interconnect architecture is an open standard which addresses the needs of a variety of embedded infrastructure applications. Applications include interconnecting microprocessors, memory, and memory mapped I/O devices in networking equipment, storage subsystems, and general purpose computing platforms. This interconnect is intended primarily as an intra-system interface, allowing chip to chip and board to board communications at giga-byte per second performance levels. Supported programming models include globally shared distributed memory and message passing. In its simplest form, the interface can be implemented in an FPGA end point. The interconnect architecture defines a protocol independent of a physical implementation. The physical features of an implementation utilizing the interconnect are defined by the requirements of the implementation, such as I/O signaling levels, interconnect topology, physical layer protocol, error detection, etc. The interconnect is defined as a layered architecture which allows scalability and future enhancements while maintaining compatibility.

## Introduction

Computer and embedded system development continues to be burdened by divergent requirements. On one hand the performance must increase at a nearly logarithmic rate, while on the other hand system cost must stay the same or decrease. Several applications, such as those found in networking and telecommunications infrastructure equipment, are also burdened with increasing capabilities while decreasing the board size and ultimately the floor space which equipment occupies.

The connection fabric between microprocessors and peripherals has traditionally been comprised of a hierarchy of shared buses (Figure 1). Devices are placed at the appropriate level in the hierarchy according to the performance level they require. Low performance devices are placed on lower performance buses, which are bridged to the higher performance buses so as to not burden higher performance devices. Bridging is also done to address legacy interfaces.

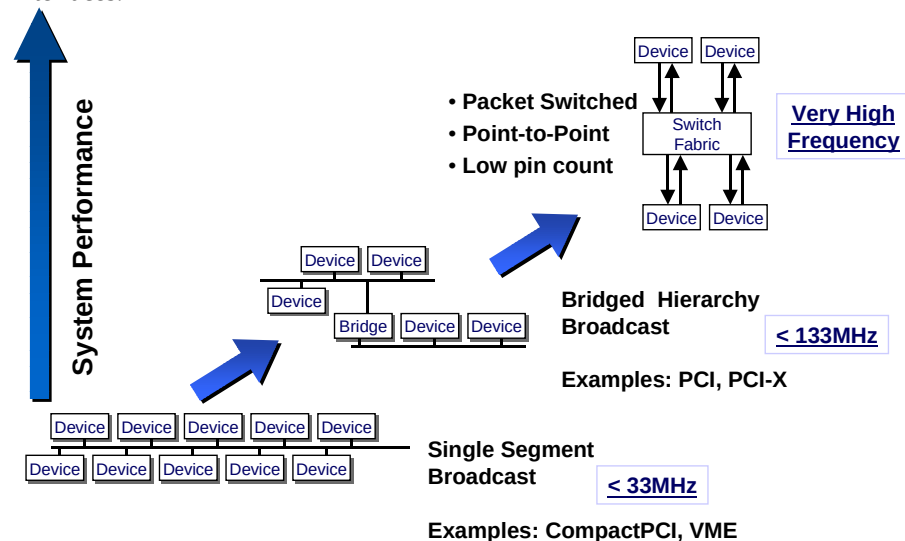


Figure 1 Higher system performance levels require adoption of point to point switched interconnects

The need for higher levels of bus performance is driven by two key factors. First, the need for raw data bandwidth to support higher peripheral device performance demands. Second the need for more system concurrency. Overall system performance has increased due to the application of distributed DMA and processing.

## System topology trends

Over the past several years the shared multi-drop bus has been exploited to its full potential. Many techniques have been applied, such as increasing frequency, widening the interface, pipelining transactions, splitting transactions, and allowing out of order completion. Continuing to work with a bus in this manner creates several design issues. Increasing bus width, for example, reduces the maximum achievable frequency due to skew between signals. More signals will also result in more pins on a device, resulting in a higher product cost and a reduction in the number of interfaces the device can provide.

Another issue is the desire to increase the number of devices that can communicate directly with each other. As frequency and width increase, the ability to have more than a few devices

attached to a shared bus becomes a difficult design challenge. In many cases, system designers have inserted a hierarchy of bridges to reduce the number of loads on a single bus.

Developed as an open standard, the *RapidIO* architecture addresses the needs of present and future systems. The *RapidIO* architecture is for a point-to-point, packet-switched interconnect. *RapidIO* is focused as a processor, memory, and memory mapped I/O interface optimized for use inside a chassis. In embedded system applications *RapidIO* has limited to no impact on the software infrastructure that operates over it. It can be implemented in few gates, offers low operation latency and high bandwidths.

Network equipment is usually separated into what is commonly called a control plane and a data-forwarding plane as shown in Figure 2. The data-forwarding plane is responsible for moving data packets through the machine while the control plane manages the data movement. *RapidIO* provides all of the necessary attributes to be useful in the control plane.

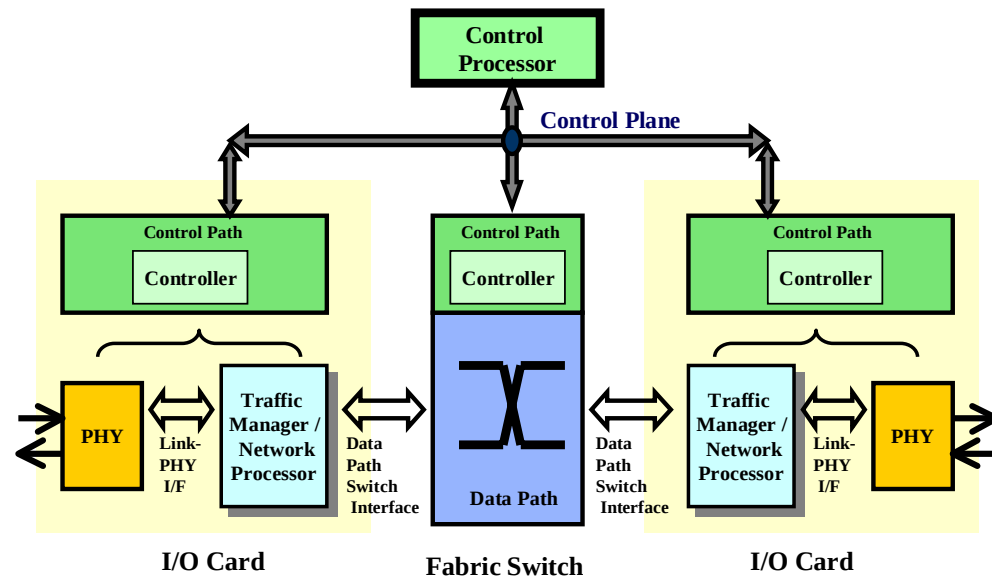


Figure 2 *RapidIO* is targeted toward control plane applications in network equipment

*RapidIO* can be categorized as being an intra-system interconnect as shown in Figure 3. There are similar movements in the industry away from shared buses, towards fabrics. Proposed fabrics optimized for I/O include InfiniBand<sup>1</sup>.

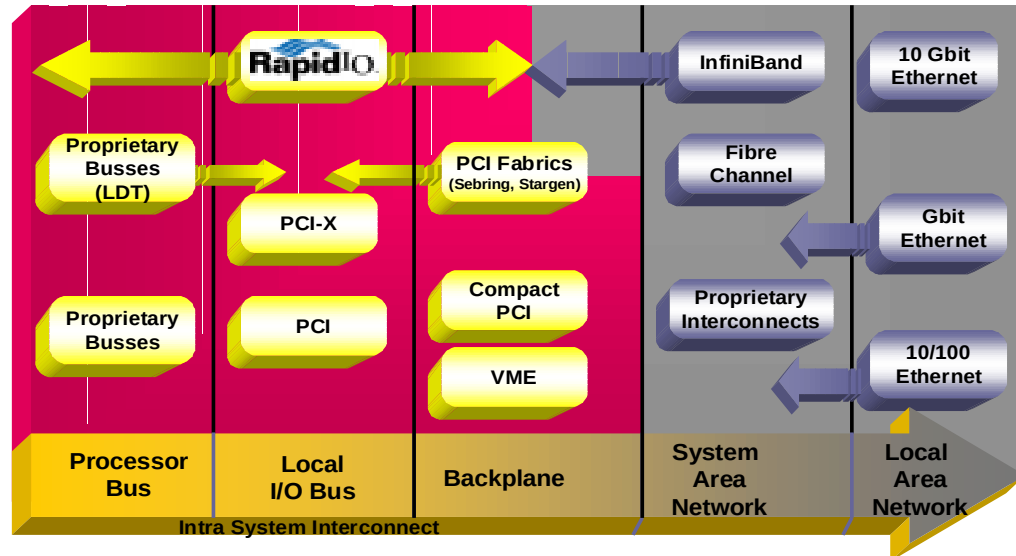


Figure 3 RapidIO connects processors, memory, and peripherals within a subsystem or across a backplane.

InfiniBand is targeted as a System Area Network (SAN) interconnect. A SAN is used to cluster systems together to form larger highly available systems. SANs usually connect whole computers together within distances of up to 30 meters. Operations through a SAN are typically handled through software drivers using message channels or remote direct memory access (RDMA).

*RapidIO* differs from InfiniBand in that it is targeted toward embedded, in the box communications. *RapidIO* can be thought of as the mechanism to connect devices together to form the computer system. InfiniBand connects these computer systems together to form the SAN.

### System Applications

The *RapidIO* interconnect is targeted as a device level interface for use in environments where multiple devices must work in a tightly coupled load/store environment. Figure 4 illustrates a generic system containing memory controllers, processors, and I/O bridges connected using *RapidIO* switches.

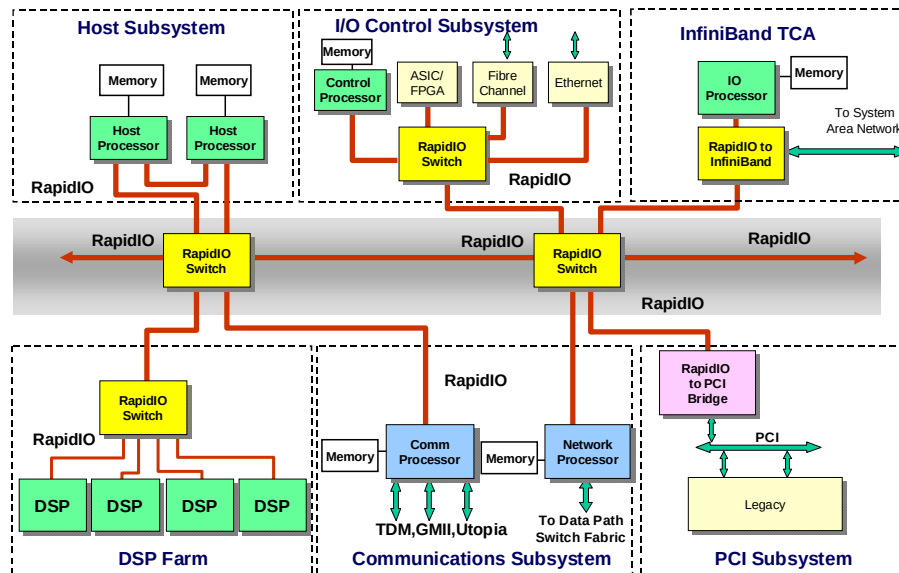


Figure 4 RapidIO allows a variety of devices to be interconnected

In computing, the PCI bus<sup>2</sup> is ubiquitous. Enterprise storage applications, for example, use PCI to bridge multiple disk channels to a system. As disk throughput has increased so has the need for higher system throughput (a la PCI-X). To meet the electrical requirements of higher bus frequencies the number of devices per bus segment has decreased. Therefore to connect the same number of devices, more bus segments are required. These applications require higher bus performance, more device fan-out, and greater device separation. PCI-to-PCI bridge devices could be used to solve this problem but this would come at the cost of many pins and a limit in the total transmission distance between devices.

RapidIO can be used for transparent PCI-to-PCI bridging allowing for a flattened architecture utilizing fewer pins with greater transmission distances. Figure 5 shows one such bridged system. In this example, several PCI-X bridges are connected together using RapidIO switches. An InfiniBand Channel Adapter can be provided if the system is to become part of a wider System Area Network.

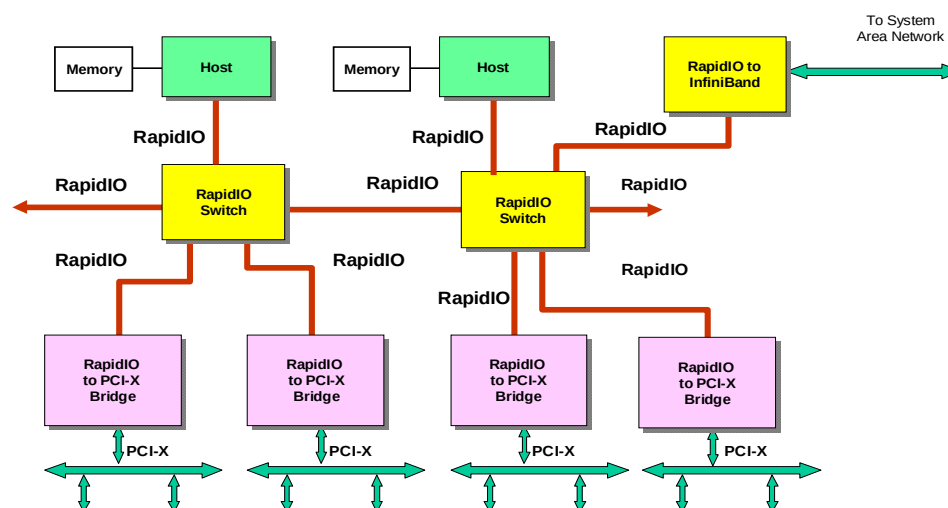


Figure 5 RapidIO as a PCI bridging fabric reduces device pin count

Many systems require the partitioning of functions into field replaceable units. These printed circuit boards have traditionally been interconnected using multi-drop interfaces like VME or Compact PCI. Higher system level performance can be achieved by using *RapidIO*. For hot swap applications, the removal of a device has no electrical impact on other devices as it would in a shared bus environment.

## Philosophy

The architecture of *RapidIO* was driven with certain principal objectives:

- **Focus on applications that connect devices which are within the box or chassis:** As an example, since devices are not intended to be connected with several meters of cable the interface can use a simple signaling and flow control scheme.
- **Limit the impact on software:** Many applications are built on a huge legacy of memory mapped I/O. In such applications the interconnect must not be visible to software. Offering more abstracted interfaces requires a large software re-engineering effort.
- **Confine protocol overhead:** Because of the typical application dependence on latency and bandwidth it is important that the protocol use no more transaction overhead than is absolutely necessary.
- **Partition the specifications to limit the necessary function set to only those needed for the application:** This limits design complexity and enables future enhancements without impacting the top to bottom specification.
- **Manage errors in hardware:** In meeting the high standards of availability, yet limiting the impact to performance and software infrastructure, it is important that the interconnect be able to detect errors. The transmission media should have the capability of detecting multiple bit errors. Going a step further the interconnect must utilize hardware mechanisms to survive single bit errors and many multiple bit errors.
- **Limit silicon footprint:** Many applications require the use of a high performance interconnect but have a limited transistor budget. It is important that the interface be able to fit within a small transistor count. As an example, it is important that it fit within a commonly available FPGA technology.
- **Build from common process technology I/O:** Since this interface is targeted for intra-system communications, it should not require separate discrete physical layer hardware. This means that the I/O technology should be power efficient and limit the use of exotic process technology. Further, there interface will leverage industry standard I/O technology and the support infrastructure associated with such standards.
- **Choose a memory coherency scheme optimized for small (16 or fewer caching devices) distributed memory systems:** Most applications of shared memory usually involve only few processing elements.

*RapidIO* is specified in a 3-layer architectural hierarchy (Figure 6). A Logical specification defines the overall protocol and packet formats. This is the information necessary for end points to initiate and complete a transaction. A Transport specification provides the necessary route information for a packet to move from end point to end point. A Physical specification

contains the device level interface such as packet transport mechanisms, flow control, electrical characteristics, and low level error management.

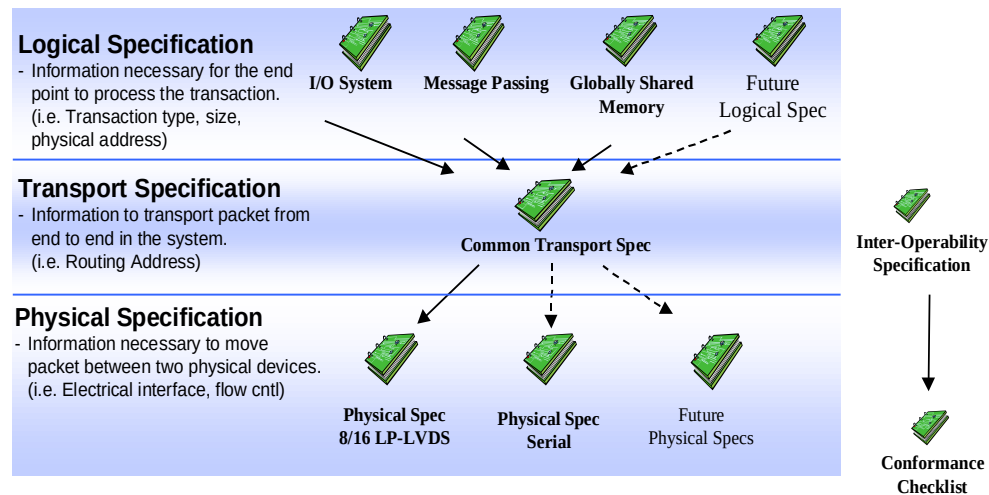


Figure 6 The RapidIO specification hierarchy allows flexibility in usage and future specification development

This partitioning provides the flexibility to add new transaction types to a logical specification without requiring modification to the transport or physical layer specifications.

The *RapidIO* feature set and protocols are based upon a number of considerations for both general computing and embedded applications. These considerations are broken down into three categories: functional, physical, and performance.

### Functional Considerations

The *RapidIO* architecture is targeted toward memory mapped distributed memory systems and subsystems. A message passing programming model is supported as well as a globally shared distributed memory programming model. This enables both general purpose multiprocessing and distributed I/O processing to co-exist under the same protocol.

Message passing and DMA devices can improve the interconnect efficiency if larger data payloads are encapsulated within a single packet, so *RapidIO* supports a variety of data sizes within the packet formats. Since the message passing programming model is fundamentally a non-coherent non-shared memory model, *RapidIO* assumes that processors accessing common memory space use software maintained coherency methods. This approach is commonly used today in many embedded distributed processing applications.

For the globally shared memory programming model, a directory based coherency mechanism is chosen. This mechanism provides processing elements with all of the necessary interlocks for hardware maintained coherent memory distributed across multiple memory controllers. The *RapidIO* globally shared memory programming model is useful for sharing data structures, code, or other information without having to resort to software management of coherency.

For both programming models, reliable delivery of transactions is a requirement. This is because there is no higher level protocol (software) to guarantee the outcome of transactions as there are in more abstracted message passing protocols.

## Physical Considerations

The protocol and packet formats are independent of the topology of the physical interconnect. The protocol works whether the physical interconnect is a point-to-point, ring, bus, switched multidimensional network, duplex serial connection, etc. There is no dependency on the bandwidth or latency of the physical fabric. The protocol handles out of order packet transmission and reception. There is no requirement for geographical addressing; a device's identifier does not depend on its location in the address map, but can be assigned by other means. The physical layer specification contains the signal definitions, flow control and error management.

Initially an 8-bit and 16-bit parallel (8/16 LP-LVDS), point-to-point interface is deployed. An 8/16 LP-LVDS device interface contains a dedicated 8- or 16-bit input port with clock and frame signals, and an 8- or 16-bit output port with clock and frame signals. A source synchronous clock signal clocks packet data on rising and falling edge. The frame signal provides a control reference. Standard LVDS differential signaling is used to reduce interface complexity, provide robust signal quality, and promote good frequency scalability across printed circuit boards and connectors.

To complement the parallel interface, the RapidIO Technical Working Group is currently developing a serial physical layer.

## Performance Considerations

Packet headers are as small as possible to minimize the control overhead and are organized for fast, efficient assembly and disassembly. As the amount of data included in a packet increases, packet efficiency increases. *RapidIO* supports data payloads up to 256 bytes. Messages are very important for embedded control applications, so a variety of large and small data fields and multiple packet messages are supported.

Multiple transactions are allowed concurrently in the system, not only through the ability to pipeline transactions from a single device, but also through spatial reuse of interfaces between different devices in the system. Without this, a majority of the potential system throughput is wasted.

Transaction buffer management directly impacts the ability to pipeline multiple transactions into the system. *RapidIO* 8/16LP-LVDS devices manage packet buffer resources on a per-hop basis through a system fabric. Once the adjacent receiving device has accepted a packet the source can free up the transmission buffer and still be assured that the transaction will be delivered reliably to the target device.

Direct peer-to-peer transaction support is a must in any distributed machine. Because *RapidIO* uses source based addressing any device can communicate with any other device without regeneration of a transaction by a host controller.

Cache coherence requires a large amount of control overhead, so an interventionist (cache to cache) protocol saves a large amount of latency for memory accesses that cause another cache to provide the requested data.

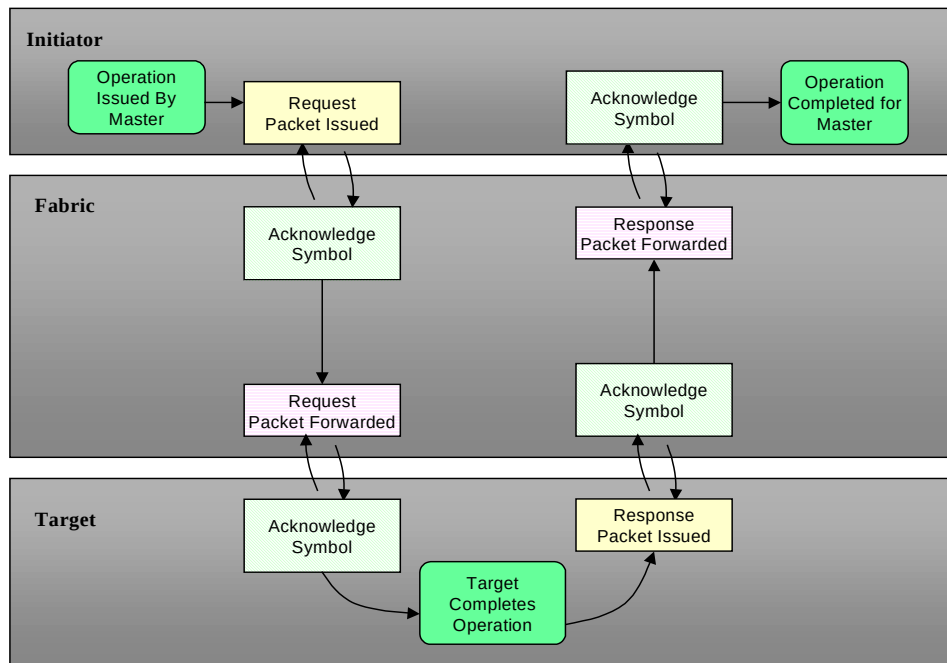
## Protocol Overview

### Packets and Control Symbols

RapidIO operations are based on request and response transactions. Packets are the communication element between end point devices in the system. A master or initiator generates a request transaction, which is transmitted to a target. The target then generates a response transaction back to the initiator to complete the operation. The RapidIO transactions are



encapsulated into packets, which include all of the necessary bit fields to ensure reliable delivery to the targeted end point. RapidIO end points are typically not connected directly to each other but instead have intervening connection fabric devices. Control symbols are used to manage the flow of transactions in the RapidIO physical interconnect. Control symbols are used for packet acknowledgement, flow control information, and maintenance functions. Figure 7 shows how transactions progress through the system.

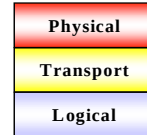


*Figure 7: Transactions are constructed with request and response packet pairs*

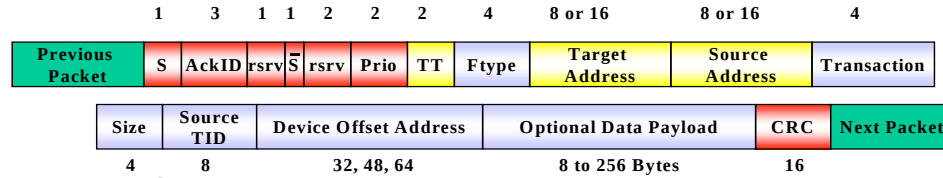
In this example, the initiator begins an operation in the system by generating a request transaction. The request packet is sent to a fabric device, which is acknowledged with a control symbol. The packet is forwarded to the target through the fabric device. This completes the request phase of the operation. The target completes request transaction and generates a response transaction. The response packet returns through the fabric device using control symbols to acknowledge each hop. Once the packet reaches the initiator and is acknowledged, the operation is considered complete.

### Packet Format

The RapidIO packet is comprised of fields from the three-level specification hierarchy. Figure 8 shows typical request and response packets. Certain fields are context dependent and may not appear in all packets.



#### Request Packets



#### Response Packets

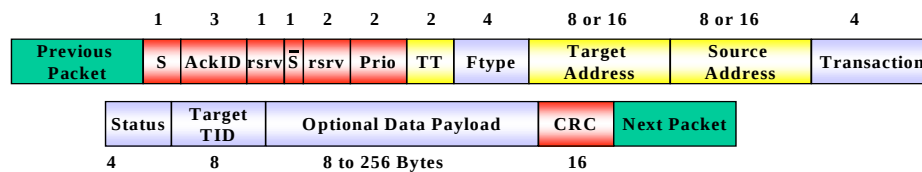


Figure 8 The RapidIO packet contains fields from the specification hierarchy.

The request packet begins with physical layer fields. The “S” bit indicates whether this is a packet or control symbol. The “AckID” indicates which packet the fabric device should acknowledge with a control symbol. RapidIO supports up to 8 unacknowledged packets between two adjacent devices. The “PRIO” field indicates the packet priority used for flow control. The “TT”, “Target Address”, and “Source Address” fields indicate the type of transport address mechanism used, the device address where the packet should be delivered, and where the packet originated. The “Ftype” and “transaction” indicate the transaction that is being requested. The “size” is an encoded transaction size. RapidIO transaction data payloads range from 1-byte to 256 bytes in size. The “srcTID” indicates the transaction ID. RapidIO devices may have up to 256 outstanding transactions between two end points. For memory mapped transactions the “Device Offset Address” follows. For write transactions a “Data Payload” completes the transaction followed by a 16-bit CRC.

Response packets are very similar to request packets. The “status” field indicates whether the transaction was successfully completed. The “TargetTID” corresponds to the request packet source transaction ID.

### Transaction Formats and Types

One of the attributes of a software transparent interconnect is the requirement for a rich set of transaction functions. The RapidIO transaction is described through two fields: the packet format type “Ftype”, and the “Transaction”. To ease the burden of transaction deciphering, transactions are grouped by format as shown in Table 1.

Table 1 RapidIO has a rich set of Transaction Formats

| Functions                         | Transaction Types                                                        |
|-----------------------------------|--------------------------------------------------------------------------|
| <b>I/O non-coherent functions</b> | NREAD (read non-sharable memory)                                         |
|                                   | NWRITE, NWRITE_R, SWRITE (write non-sharable memory)                     |
|                                   | ATOMIC (read-modify-write to non-sharable memory)                        |
| <b>Port based functions</b>       | DOORBELL (generate an interrupt)                                         |
|                                   | MESSAGE (write to port)                                                  |
| <b>System support functions</b>   | MAINTENANCE (read or write configuration, control, and status registers) |
| <b>User Defined functions</b>     | Open for application specific transactions                               |
| <b>Cache coherence functions</b>  | READ (read globally shared cache line)                                   |
|                                   | READ_TO_OWN (write globally shared cache line)                           |
|                                   | CASTOUT (surrender ownership of globally shared cache line)              |
|                                   | IKILL (instruction cache invalidate)                                     |
|                                   | DKILL (data cache invalidate)                                            |
|                                   | FLUSH (return globally shared cache line to memory)                      |
|                                   | IO_READ (read non-cacheable copy of globally shared cache line)          |
| <b>OS support functions</b>       | TLBIE (TLB invalidate)                                                   |
|                                   | TLBSYNC (force completion of TLB invalidates)                            |

## Message Passing

When data must be shared amongst multiple processing elements in a system, a protocol must be put in place to maintain ownership. In many systems, especially embedded, this protocol is often managed through software mechanisms. If the memory space is accessible to multiple parties, then locks or semaphores are used to grant access to one party. In other cases, processing elements may only have access to locally owned memory space. In these “shared nothing” machines, a mechanism is required to pass data from the memory of one processing element to another. This can be done using software visible mailbox hardware.

*RapidIO* provides a message passing extension useful in shared nothing machines. The *RapidIO* message passing logical protocol extensions describe transactions to enable mailbox and doorbell communications. A *RapidIO* mailbox is a port through which one device may send a message to another device. The receiving device controls where the message is placed when it arrives. A *RapidIO* message can consist of up to 16 packets of up to 256 bytes each for a total 4 Kbytes. A receiver can have 1 to 4 addressable message queues to capture inbound messages.

The *RapidIO* doorbell is a lightweight port-based transaction, which can be used for in-band interrupts. A doorbell message has a 16-bit software definable field, which can be used for a variety of messaging purposes between two devices.

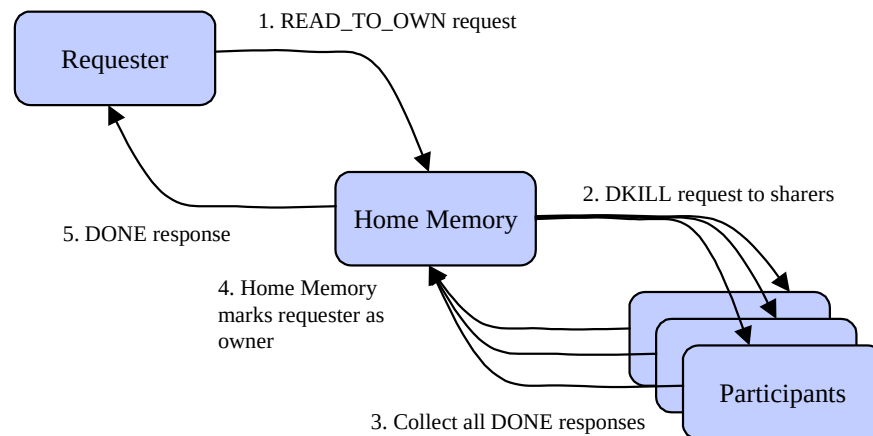
## Globally Shared Memory

One of the protocol extensions offered in *RapidIO* is support for a globally shared distributed memory system. This means that memory may be physically located in different places in the machine yet may be cached amongst different processing elements. Typically mainstream microprocessor architectures have addressed shared memory using transaction broadcasting sometimes known as bus based snoopy protocols. These are usually implemented through a centralized memory controller for whom all devices have equal or uniform access.

Massively parallel, super computers, and cluster machines, which have distributed memory systems, must use a different technique for maintaining memory coherency. A broadcast

snoopy protocol in these machines is not efficient given the number of devices that must participate and the latency and transaction overhead involved. Therefore a different mechanism is desired to keep track of where the most current copy of data resides. Such machines use hardware coherency mechanisms such as coherence directories or distributed link lists. These schemes are often referred to as cache coherent non-uniform memory access (CC-NUMA) protocols. Examples include the Stanford DASH machine<sup>3</sup>, the MIT Alewife Machine<sup>4</sup> and Scalable Coherent Interface (SCI)<sup>5</sup>.

For *RapidIO*, a directory based coherency scheme is chosen. For this method each memory controller is responsible for tracking where the most current copy of each data element resides in the system. A directory entry is maintained for each device participating in the coherency domain. Simple coherency states of Modified, Shared, or Local (MSL) are tracked for each element. Figure 9 shows an example of a “read with intent of modification” request to a memory controller. For this example other devices in the system have shared copies of the data. The memory controller indexes the requested data in its directory and subsequently issues appropriate invalidation transactions to the sharers. At the completion of this set of operations the memory controller forwards the latest copy of the data to the requestor to complete the transaction.



*Figure 9 the memory controller is responsible for managing coherency in the directory based scheme.*

To reduce the directory overhead required, the architecture is optimized around small clusters of 16 or fewer processors known as coherency domains. With the concept of domains, it is possible for multiple coherence groupings to coexist in the interconnect as tightly coupled processing clusters.

### Future Extensions

*RapidIO* is partitioned to support future protocol extensions. This can be done at the user level through the application of user definable transaction format types, or through future of reserved fields. *RapidIO* is architected so that switch fabric devices do not need to interpret packets as they flow through, lending itself toward forward compatibility.

### Flow Control

Flow control is an important aspect of any interconnect. The goal is for devices to be able to complete transactions in the system without being blocked by other transactions. Bus based interconnects use arbitration algorithms to be sure that devices make forward progress and that urgent transactions take precedence over less urgent ones. With switch based interconnects, transactions enter the interconnect at different points in the system and there is

no centralized arbitration mechanism. This creates the need for alternate methods to manage transaction flow.

One of the objectives of *RapidIO* is to limit overhead and complexity as much as possible especially in the area of flow control. For *RapidIO*, flow control is specified as part of the physical specification. This is because transaction flow is largely dependent on the physical interconnect and system partitioning. Each *RapidIO* 8/16 LP-LVDS packet carries with it a transaction priority. Each transaction priority is associated with a transaction request flow. There currently are three transaction request flows defined. Transaction request flows allow higher priority requests to move ahead of lower priority requests. Transaction ordering rules are managed within a transaction request flow but not between flows.

Transaction request flows make it possible for system implementers to structure traffic through the machine in such a way as to guarantee deterministic behavior. For example, an application may require that certain time critical operations be delivered through the system within a guaranteed period. This data can be placed at a higher priority request flow. To handle critical data, the switch fabric may contain queuing structures that move higher priority traffic ahead of the lower priority. These queuing structures may include interval timers to guarantee that flows from different ports are getting targeted to a common port are given opportunity to move critical data through. The results are isochronous delivery behavior of certain data classes.

*RapidIO* also describes three types of flow control mechanisms; retry, throttle, and credit based. The retry mechanism is the simplest mechanism and is required not only for flow control but also as a component of hardware error recovery. A receiver, unable to accept a packet because of a lack of resources or because the packet was corrupt, may respond with a retry control symbol. The sender will retransmit the packet.

The throttle mechanism makes use of the idle control symbol. Idle control symbols may be inserted into a packet during packet transmission. This allows a device to insert wait-states in the middle of packets. A receiving device can also send a throttle control symbol to the sender requesting that it slow down by inserting idle control symbols.

The credit-based mechanism is useful for devices that implement transaction buffer pools, especially fabric devices. In this scheme certain control symbols contain a buffer status field which represents the current status of the receiver's buffer pool for each transaction flow. A sender only sends packets when it knows the receiver has a buffer available to store it to. This information enables the switch fabric device to discover downstream blocking conditions and to make packet output selections.

## Physical Interface

The *RapidIO* logical specification is defined to be physical layer independent. This means that the *RapidIO* protocol could be transmitted over anything from serial to parallel interfaces, from copper to fiber media. The first physical interface considered and defined is known as the 8- or 16-bit link protocol end point specification (8/16 LP-LVDS). This specification is defined as having 8 or 16 data bits in each direction along with clock and frame signals in each direction (Figure 10.)

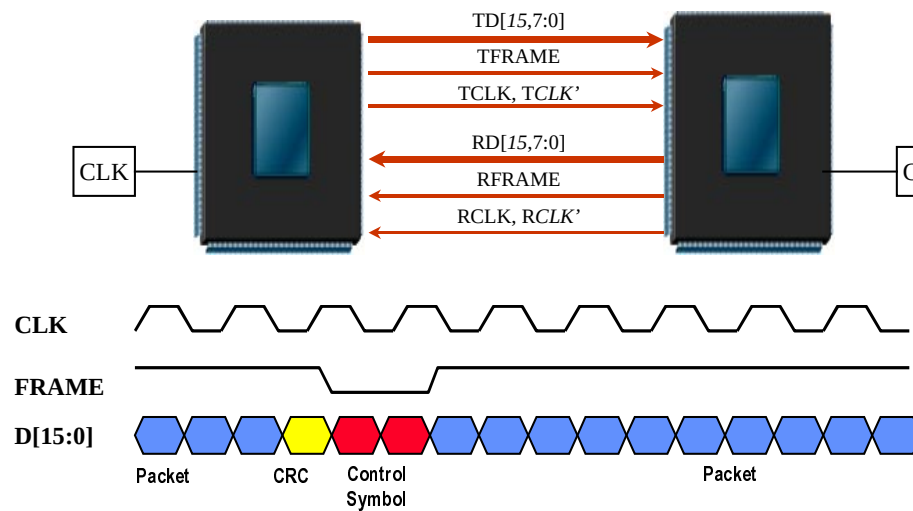


Figure 10 the RapidIO 8/16 LP-LVDS physical layer provides for full duplex communications between devices.

8/16 LP-LVDS is a source synchronous interface which means that a clock is transmitted with the associated data. Source synchronous clocking allows longer transmission distances at higher frequencies. Two clock pairs are provided for the 16-bit interface to help control skew. The receiving logic is able to use the receive clock for re-synchronization of the data to its local clock domain.

The FRAME signal is used to delineate the start of a packet or control symbol. It operates as a no return to zero (NRZ) signal where any transition marks an event.

### Electrical Interface

RapidIO adopts the IEEE 1596.3 Low Voltage Differential Signals (LVDS)<sup>6</sup> standard as basis for the electrical interface. LVDS is a low swing (250 to 400 mV) constant current differential signaling technology targeted toward short distance printed circuit board applications. LVDS is technology independent and can be implemented in CMOS. Differential signals provide improved noise margin, immunity to externally generated noise, lower EMI, and reduced numbers of power and ground signal pins. LVDS has a simple receiver based termination of 100 ohms. At higher frequencies the RapidIO specification recommends additional source termination to reduce signal reflection effects.

The target frequencies of operation are from 250MHz to more than 1GHz. Data is sampled on both edges of the clock. The resulting bandwidth scales to 4 GByte/sec for the 8-bit and 8 GBytes/sec for the 16-bit interfaces.

## Maintenance and Error Management

RapidIO steps beyond traditional bus based interfaces by providing a rich set of maintenance and error management functions. These enable the initial system discovery, configuration, error detection and recovery methods. A discovery mechanism with certain similarities to PCI is used to find devices in the system and configure them.

## Maintenance

A maintenance operation is supported by all *RapidIO* devices and is used to access predefined maintenance registers within each device. The registers contain information about the device, including its capabilities and memory mapped requirements. Also included are error detection and status registers such as watchdog timer settings. For more complex multi-bit errors, software may make use of these registers to recover or shut down a *RapidIO* device.

## System Discovery

Through the use of the maintenance operation, a *RapidIO* host may traverse a system and configure each device in the network. Since *RapidIO* allows complex system topologies like meshes, a mechanism is provided so that the host can determine if it has been to a node before. For high reliability applications, *RapidIO* allows more than one host to configure the machine.

## Error coverage

The mean time between failure (MTBF) and mean time to repair (MTTR) of a system are often critical considerations in embedded infrastructure applications. *RapidIO* applications require reliable packet delivery. It is intolerable for an error to go undetected or for a packet to be dropped. Further, once detected, it is desirable to recover from these errors with minimal system interruption. Because of these factors and since *RapidIO* operates at very high frequencies, it is necessary to provide strong error coverage. Much of the *RapidIO* error coverage is handled through the physical layer specification. This allows different coverage strategies depending on the physical environment without affecting the transport and logical specifications.

The 8/16 LP-LVDS physical layer can detect moderate burst errors. Because this physical layer is intended as a board level interconnect, it is assumed that the interface will not suffer from a high bit error rate as it might if it were traversing a cable. Several error detection schemes are deployed to provide coverage for 8/16 LP-LVDS. Packets are covered using a CCITT16 cyclic redundancy check (CRC), and control symbols are covered through transmission of the control symbol and its complement. Packets are transmitted and acknowledged following a strict sequence using the ACKID field. The FRAME signal is covered against inadvertent transitions by treating it as a no-return-to-zero NRZ signal. It must stay in the new state for more than 1 clock to be considered valid.

## Error Recovery

The control symbol is at the heart of the hardware error recovery mechanism. Should a packet be detected with a bad checksum, control symbols are sent to verify that both sender and receiver are still synchronized and a packet retry is issued. Should a transaction be lost, a watchdog time-out occurs and the auto recovery state machines attempt to re-synchronize and retransmit.

If an interface fails severely, it may not be able to recover gracefully in hardware. For this extreme case, *RapidIO* hardware can generate interrupts so that system software can invoke a higher level error recovery protocol. Software may query the maintenance registers to reconstruct the current status of the interface and potentially restart it. An in-band device reset control symbol is provided.

## Performance

*RapidIO* is intended as a processor and memory interface where both latency and bandwidth must be considered. *RapidIO* may also be used to carry data streams in which deterministic delivery behavior is desired. Separate clock and frame signals are provided to eliminate

encoding and decoding latency. Source routing and transaction priority tagging limit blocking of packets, especially those of a critical nature. Large data payloads of up to 256 bytes and response-less stream write operations move larger volumes of data with less transaction overhead.

### **Packet Structures**

The *RapidIO* packet is structured to promote simplified construction and parsing of packets in a wider on-chip parallel interface, limiting the amount of logic operating on the narrower high frequency interface. Packets are organized in byte granularities and further in 32-bit word alignments. In this way *fields* land consistently in specific byte lanes on the receiving device, limiting the need for complex assembly and parsing logic.

### **Source Routing and Concurrency**

Traditional bus-based systems, such as those using PCI, have relied on address broadcasting to alert targets of an operation. This is effective since all devices monitor a common address bus and respond when they recognize a transaction to their address domain. Unfortunately, only one master can be broadcasting an address at a time.

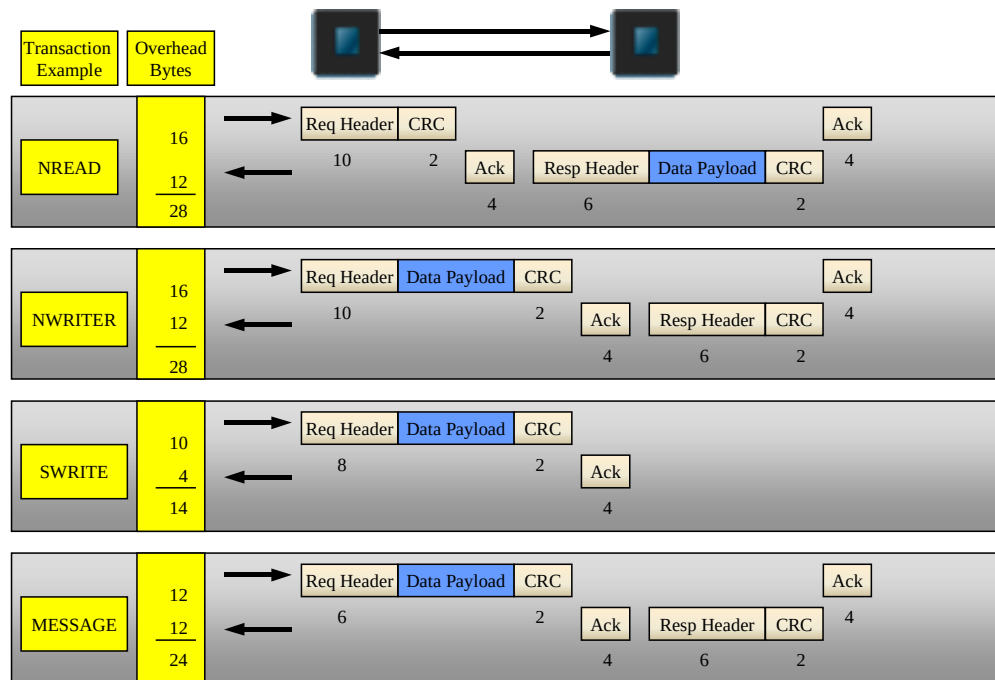
Switch-based systems can employ two methods to route transactions, broadcast or source routing. For the broadcast scheme a packet is sent to all connected devices. It is expected that one and only one device actually respond to the packet. The advantage of the broadcast scheme is that the master does not need to know where the receiver resides. The disadvantage is that there is a large amount of system bandwidth wasted on the paths for which the transaction was not targeted.

To take full advantage of available system bandwidth, *RapidIO* employs source routing. This means that each packet has a source specified destination address which instructs the fabric specifically where the transaction is to be routed. With this technique only the path between the sender and receiver is burdened with the transaction. The switch devices make decisions on a much small number of bits resulting in small route tables and lower latency. This method leaves open bandwidth on other paths for other devices in the system to communicate with each other concurrently. This scheme does not preclude the use of broadcast or multicast routing.

### **Packet Overhead**

A performance concern in any tightly coupled intra-system interconnect is the transaction overhead required for the interface. Such overhead includes all bytes sent to complete a transaction such as arbitration, addresses, acknowledgements, error coverage, etc. Figure 11 shows some typical *RapidIO* operations and the number of bytes required to complete each operation. It is important to remember that *RapidIO* is a full duplex interface and therefore the interface can be fully pipelined with several outstanding operations at various stages of completion. Reply overhead does not contend with sending overhead. This is different from traditional buses, which require turnaround cycles and arbitration phases that add to the overhead.





Notes: Assumes 8-bit Route and 32-bit physical offset address

Figure 11 Operation overhead includes the total bytes sent in each direction to complete an operation.

The *RapidIO* 8/16LP-LVDS specification presently defines four operating frequencies and two widths. Table 2 shows the operating points for *RapidIO* and sustained full duplex data bandwidth.

| Configuration | 8-bit Mode |                         |                          | 16-bit Mode |                         |                          |
|---------------|------------|-------------------------|--------------------------|-------------|-------------------------|--------------------------|
|               | PEAK       | Sustained<br>32 byte Op | Sustained<br>256 byte Op | PEAK        | Sustained<br>32 byte Op | Sustained<br>256 byte Op |
| 125Mhz        | 4Gb/s      | 2Gb/s                   | 3.7Gb/s                  | 8Gb/s       | 4Gb/s                   | 7.5Gb/s                  |
| 250Mhz        | 8Gb/s      | 4Gb/s                   | 7.5Gb/s                  | 16Gb/s      | 8Gb/s                   | 15Gb/s                   |
| 500Mhz        | 16Gb/s     | 8Gb/s                   | 15Gb/s                   | 32Gb/s      | 16Gb/s                  | 30Gb/s                   |
| 1Ghz          | 32Gb/s     | 16Gb/s                  | 30Gb/s                   | 64Gb/s      | 32Gb/s                  | 60Gb/s                   |

Sustained Operations include all transaction overhead and assume fully loaded full duplex communications  
 32-byte Operation is 50% Efficient  
 256-byte Operation is >90% Efficient

For Unidirectional traffic 32-byte is >70% Efficient,  
 256-byte is >95% Efficient

Table 2 the 8/16LP-LVDS physical layer can scale to the needs of the application

## Operation Latency

In load/store environments, where processors or other devices are dependent on the results of an operation before proceeding, latency becomes a key factor to system performance. It is assumed that the operations of interest here are small byte or cache-line oriented operations. Since *RapidIO* is narrower than traditional parallel buses, an operation requires more clock cycles for data transmission; also *RapidIO* has extra overhead associated with routing and error management. However, *RapidIO* has limited to no contention or arbitration, higher operating frequency, and a concurrent reply path.

Some examples (Figure 12) are presented to illustrate estimated latency through a typical system. For the examples both 8-byte and 64-byte reads are considered. The latency numbers include the time to assemble, transmit, receive, synchronize, and disassemble an operation. The first example shows a traditional processor mezzanine bus where the processor is fetching data from a memory controller or from a PCI peripheral. The second example shows a CPU fetching data from a PCI peripheral using a *RapidIO* to PCI bridge device. The third example shows a processor fetching data from a PCI peripheral and a *RapidIO* peripheral through a *RapidIO* switch. It is noted that a store and forward switch is considered here and performance improvements could be made using a cut-through switch. The reader is cautioned in comparing these numbers to traditional interfaces since the sequencing hardware necessary to generate and receive a transaction is not usually accounted for. *RapidIO* latency is very comparable to traditional bus latencies. In busier systems, if devices are not contending for a common resource, *RapidIO* latency should be lower since devices would not be required to arbitrate for a common bus interface. Typically latency is dominated by DRAM access times and not by the system interconnect.

| Example | Description                                    | Latency (ns) |              |
|---------|------------------------------------------------|--------------|--------------|
|         |                                                | 8 byte Read  | 64 byte Read |
| 1       | Classic CPU Bus (133MHz) to Memory             | 60           | 120          |
| 2       | RapidIO16 (1GHz) to Memory                     | 60           | 70           |
| 3       | RapidIO8 (500MHz) to Memory                    | 80           | 140          |
| 4       | Classic CPU (133MHz) Bus to PCI Periph         | 110          | 520          |
| 5       | RapidIO16 (1GHz) to PCI Periph                 | 140          | 320          |
| 6       | RapidIO8 (500MHz) to PCI Periph                | 160          | 390          |
| 7       | RapidIO8 (500MHz) to Periph through Switch     | 170          | 280          |
| 8       | RapidIO8 (500MHz) to PCI Periph through Switch | 250          | 530          |

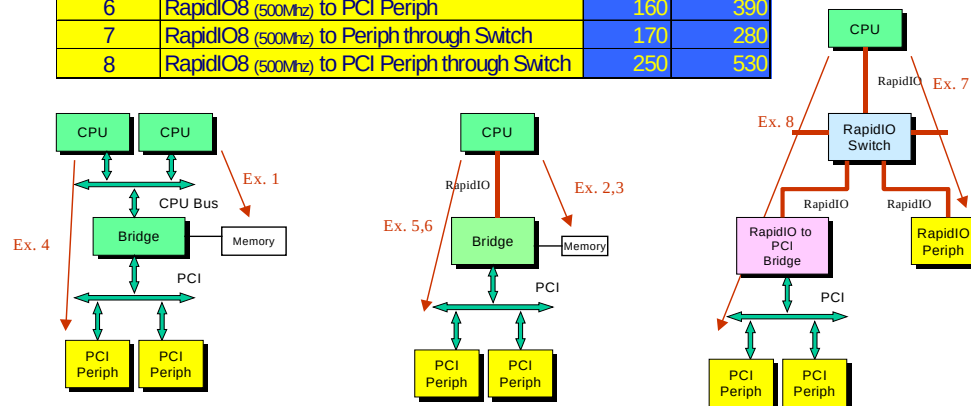


Figure 12 Latency is considered from the assembly to extraction of the packet.

## Summary

The *RapidIO* interconnect provides a robust packet-switched system level interconnect. It is a partitioned architecture that can be further enhanced in the future. It enables higher levels of system performance while maintaining or reducing the implementation costs. A *RapidIO* end point can be implemented in a small silicon footprint. A proven industry standard signaling scheme (LVDS) is used for the first physical interface. Error management includes the ability to detect multi-bit errors and survive most multi-bit and all single bit errors. Protocol overhead and latency are very comparable to current bus technologies.

<sup>1</sup> InfiniBand Trade Association, 5440 SW Westgate Dr, Suite 217, Portland OR, 97221.

<sup>2</sup> "PCI Local Bus Specification, Rev.2.2," PCI Special Interest Group 1999.

<sup>3</sup> D. Lenoski, et. al., "The DASH Prototype: Implementation and Performance," Computer Systems Laboratory, Stanford University.

---

<sup>4</sup> A. Agarwal, et. al. "The MIT Alewife Machine: Architecture and Performance," International Symposium on Computer Architecture, 1995.

<sup>5</sup> "IEEE Std. 1596-1992, IEEE Standard for Scalable Coherent Interface (SCI)," IEEE Computer Society, 1992.

<sup>6</sup> "IEEE Std. 1596.3-1996, IEEE Standard for Low-Voltage Differential Signals (LVDS) for Scalable Coherent Interface (SCI)," IEEE Computer Society, 31 July 1996.